

G-Contour: GPU Accelerated Contour Tracing For Large-Scale Layouts

Shuo Yin¹, Jiahao Xu¹, Jiayi Jiang¹, Mingjun Li¹, Yuzhe Ma², Tsung-Yi Ho¹, Bei Yu¹

¹The Chinese University of Hong Kong

²Hong Kong University of Science and Technology (Guangzhou)

Abstract—Contour tracing is a fundamental operation in computer vision and image processing, with applications ranging from object recognition to shape analysis. In the field of electronic design automation (EDA), contour tracing plays a critical role in layout processing tasks such as lithography simulation and mask optimization. In this paper, we present G-Contour, the first GPU-accelerated contour tracing framework, designed to efficiently handle large-scale layouts. G-Contour incorporates several GPU-accelerated image processing algorithms based on parallel geometry techniques to achieve significant speedups over traditional CPU-based methods. We evaluate G-Contour in real-world VLSI applications involving large-scale layout processing. The experimental results demonstrate that G-Contour achieves a speedup of over 90 $\times$ compared to state-of-the-art CPU-based contour tracing frameworks. Moreover, G-Contour could be a versatile tool that extends beyond VLSI applications, with potential applicability in various domains of computer vision and image processing, making it a valuable resource for both researchers and practitioners.

I. INTRODUCTION

An object’s contour, defined as the boundary separating it from its background, provides critical information for object representation and image recognition. Contours enable fundamental tasks such as segmenting objects, calculating their sizes, classifying shapes, and identifying feature points based on contour properties like length and curvature. Furthermore, contour information facilitates shape preservation and restoration, supporting diverse applications across various domains.

Contour tracing [1], [2] is a fundamental problem in image processing and computer graphics, with diverse applications including object detection [3] and image segmentation [4]. In the context of Electronic Design Automation (EDA), contour tracing plays a critical role in various layout processing tasks. For example, in semiconductor manufacturing, Optical Proximity Correction (OPC) [5]–[7] is a critical step for improving the resolution of photolithography masks. Inverse Lithography Technology (ILT) [8]–[11], one of the most advanced OPC techniques, employs a pixel-based approach. It rasterizes the mask, originally represented in a polygon format (e.g., GDSII), into an image and optimizes it by manipulating pixel values. Following the optimization process, the mask is converted back to the GDSII format through contour tracing, effectively the inverse operation of rasterization [12]. This conversion enables downstream mask rule checking (MRC) [13] and manufacturing processes.

Contour tracing also plays a vital role in processing the outputs of lithography simulation within the OPC workflow. The simulation process models the optical projection of the

mask, accounting for light diffraction effects, to generate a predicted on-wafer printed image. Contour tracing is then employed to convert this simulated image into a vector representation. Notably, commercial lithography simulation tools, such as Mentor Graphics [14], often utilize Scalable Vector Graphics (SVG) to store the resulting contours. This vector format offers advantages such as reduced storage requirements compared to large raster images and facilitates visualization and manipulation using layout editing tools like KLayout [15].

GPU-accelerated computing is now widely adopted for ILT [9] and lithography simulation [16] due to its parallel processing capabilities suitable for scientific computing. Consequently, mask images and simulation results are frequently processed directly within GPU memory. However, contour tracing is typically executed on the CPU, creating a significant performance bottleneck in the overall workflow. This bottleneck arises from the necessity of transferring tensor data between GPU and CPU memory, compounded by the limited parallelism inherent to CPUs. Therefore, accelerating contour tracing on the GPU is crucial for enhancing the performance of OPC workflows and shortening the time-to-market for semiconductor products.

Motivated by the challenges above, we propose *G-Contour*, the first GPU-based contour tracing framework capable of handling large layouts for modern semiconductor manufacturing. G-Contour incorporates a union-find-based parallel component analysis algorithm to identify pattern contours and employs a search-based method to extract these contours in polygon format. Experimental results on real-world benchmarks for common design for manufacturing (DFM) tasks demonstrate that G-Contour achieves significant speedup compared to state-of-the-art CPU-based contour tracing frameworks. As a general-purpose contour tracing framework, G-Contour’s applicability extends beyond EDA to broader domains such as image processing and computer graphics. The main contributions of G-Contour are as follows:

- We introduce G-Contour, the *first* GPU-accelerated framework addressing the contour tracing problem for large-scale EDA layouts.
- G-Contour incorporates a novel parallel component analysis algorithm and an efficient search-based method for contour extraction on GPUs.
- Experimental results show that G-Contour achieves significant speedups over the state-of-the-art CPU implementation (OpenCV), reaching up to 10 $\times$ on average and 98 $\times$

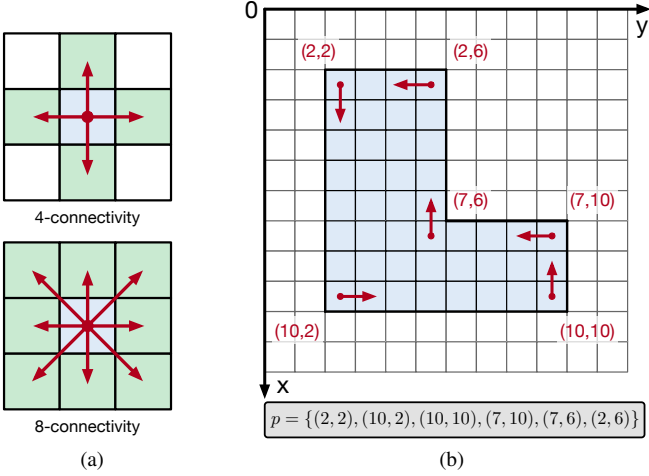


Fig. 1 Combined representation of connectivity and polygon illustration: (a) Illustration of 4-connectivity and 8-connectivity; (b) Example of polygon representation.

in peak cases.

- Beyond its primary application in EDA, G-Contour serves as a general-purpose tool applicable to broader tasks in image processing and computer graphics.

II. BACKGROUND

This section defines the contour tracing problem and introduces the fundamental concepts relevant to this work.

A. Image Connectivity

An image I is represented on a two-dimensional lattice, where each point (x, y) corresponds to a pixel denoted as $I(x, y)$. Within this pixel grid, two types of connectivity are defined: 4-connectivity and 8-connectivity.

- **4-connectivity:** Two pixels $I(x_1, y_1)$ and $I(x_2, y_2)$ are 4-connected if they are adjacent in the horizontal or vertical direction, i.e., $|x_1 - x_2| + |y_1 - y_2| \leq 1$.
- **8-connectivity:** Two pixels $I(x_1, y_1)$ and $I(x_2, y_2)$ are 8-connected if they are adjacent in any direction, including diagonals, i.e., $\max(|x_1 - x_2|, |y_1 - y_2|) \leq 1$.

Visually, as illustrated in Fig. 1(a), considering pixels as squares, 4-connected pixels share edges, whereas 8-connected pixels share either edges or vertices.

B. Geometry Representation of Polygon

A polygon is a closed planar shape defined by an ordered sequence of vertices connected by line segments, referred to as edges. Vertices represent the points where edges intersect, while edges are the segments connecting consecutive vertices.

Polygons defined using 4-connectivity are restricted to horizontal and vertical edges, forming a Manhattan geometry. In contrast, 8-connectivity permits edges at 45-degree angles, offering greater flexibility in shape representation.

A polygon can be described as an ordered list of vertices as shown in Equation (1), arranged either in a clockwise or counterclockwise direction.

$$p = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}, \quad (1)$$

where (x_i, y_i) are the coordinates of the vertices and n is the number of vertices in the polygon. In GDSII files, a pattern is represented by its contour, which is a polygon defined by its vertices. The polygon is stored in a counterclockwise order. Throughout this paper, we use the term *contour* to refer to pattern boundary represented as polygon. Fig. 1(b) illustrates an example of a contour represented by the vertices in a counterclockwise order. We describe some properties of contour that will be referenced throughout the remainder of this paper.

Definition 1 (Foreground & Background). *The foreground is defined as the meaningful region enclosed by the contour, while the rest of the plane is referred to as the background.*

Theorem 1 (Left-Hand Rule). *For a contour represented in a counterclockwise direction, the left-hand rule states that if we traverse the contour in the order of its vertices, the foreground will always be on the left side of the traversal direction.*

Theorem 1 is straightforward to understand, and Fig. 1(b) provides a visual example to illustrate it.

The polygon format can also represent patterns with complex topologies, such as those containing holes. In these cases, the contour of a hole is typically defined with an orientation opposite to that of the pattern's outer boundary. For instance, in GDSII files, the vertices defining a hole's contour are stored in clockwise order, whereas the outer boundary uses a counterclockwise order. This convention ensures adherence to the left-hand rule principle.

C. Problem Definition & Related Works

Problem 1 (Contour Tracing). *Given a layout image I , the goal of contour tracing is to identify all the contours of the patterns within the layout, represented as the set C .*

As defined in Problem 1, contour tracing is the process of identifying the outlines of patterns within a layout image to determine their shapes. This paper primarily utilizes 8-connectivity for contour tracing due to its capability to represent diagonal edges. This feature is particularly advantageous in applications such as VLSI design and image processing, where diagonal structures frequently occur.

The widely used image processing library OpenCV [17] implements contour tracing using a combination of the sweep line algorithm [18] and topological structure analysis [19]. The sweep line algorithm employs horizontal scan lines to identify the left and right edges of pattern contours. These detected edges encompass both the outer boundary and any inner contours corresponding to holes within the pattern. Subsequently, topological structure analysis connects these edges to reconstruct the complete pattern contour. While this approach exhibits memory efficiency on CPUs, requiring storage only

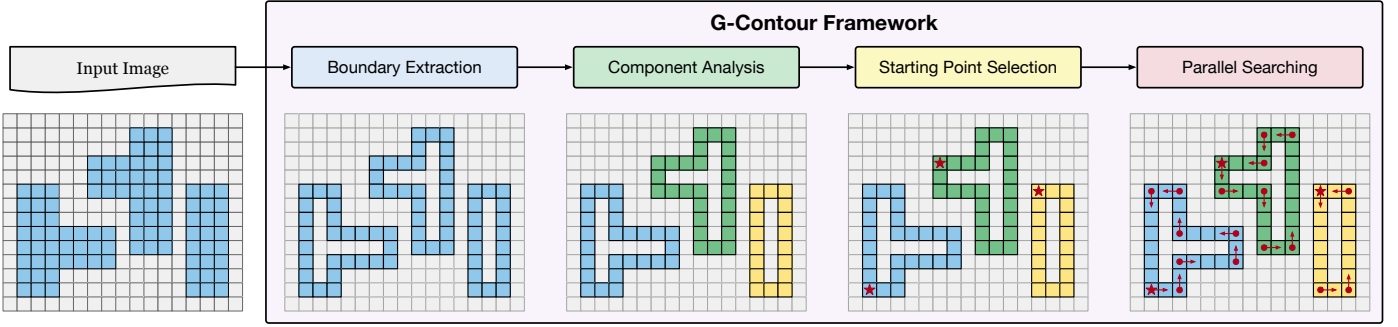


Fig. 2 The overall flow of G-Contour begins with a binary image as input and produces a set of contours represented in a counterclockwise format as output.

for the detected contour edges after the image scan, the topological analysis phase can demand significant processing time, particularly for images containing complex and dense patterns.

Search-based algorithms, such as moore-neighbor tracing [20]–[22], the radial sweep algorithm [23], and the Theo Pavlidis algorithm [24], represent widely used approaches for solving the contour tracing problem. These algorithms typically operate iteratively. For a given pattern within the image, they trace its boundary pixels in either a clockwise or counterclockwise direction, beginning from a starting pixel. Specifically, they sequentially examine adjacent foreground pixels relative to the current pixel, following a predefined directional order, and record the coordinates of the boundary pixels based on the traversal sequence. This approach allows patterns to be processed individually, making such methods inherently suitable for parallelization. Our proposed G-Contour algorithm also utilizes a search-based methodology, which is described in detail in Section IV-D.

III. OVERVIEW

Fig. 2 illustrates the overall flow of the G-Contour framework, which consists of four main components:

- ① **Boundary Extraction.** This is the first step in G-Contour, where the boundaries of each pattern in the input layout image are extracted.
- ② **Component Analysis.** In this step, the connected components in the extracted boundary are identified, serving as the basis for each individual contour.
- ③ **Starting Point Selection.** This step determines the initial vertex for each contour, which acts as the starting position for contour tracing.
- ④ **Parallel Searching.** The parallel searching step traverses each contour in parallel, ensuring efficient and simultaneous tracing of all contours.

IV. G-CONTOUR FRAMEWORK

In this section, we present the core algorithms of G-Contour, following the sequence outlined in Section III.

A. Boundary Extraction

The first step in the G-Contour framework is to extract the boundary of the layout image. The definition of the boundary is shown in Definition 2.

Definition 2 (Boundary). *The boundary of a layout image I is defined as the set of pixels that separate the foreground and background of the image.*

The boundary of a pattern differs from its contour: the boundary is defined as a set of connected pixels, whereas the contour refers to a set of points. In the rest of this paper, we use ‘boundary’ to refer to the pixels that constitute the boundary of a pattern.

The boundary image ∂I of the layout image I is defined to contain only the outer boundary or inner boundary (holes) of the pattern in I . Define function ∇f which is the transformation function between ∂I and I . The formulation of ∇f is defined as Equation (2).

$$\partial I = \nabla f(I) = \begin{cases} \partial I(x, y) < T, \forall I(x, y) \notin \mathcal{B} \\ \partial I(x, y) \geq T, \forall I(x, y) \in \mathcal{B} \end{cases} \quad (2)$$

Where T denotes the pattern threshold; for instance, for a binary image, T can be set to 0.5. $\mathcal{B}$ is a binary classification set for any pixel point $I(x, y)$ in I , determining whether $I(x, y)$ belongs to the boundary of a pattern in I . Since we are dealing with images with 8-connectivity, $\mathcal{B}$ is defined as Equation (3).

$$\mathcal{B} = \{I(x, y) \mid \exists |\delta x + \delta y| = 1, s.t. I(x + \delta x, y + \delta y) = 0\} \quad (3)$$

A pixel point $I(x, y)$ belongs to $\mathcal{B}$ if and only if, among its four neighbors (up, down, left, right), there exists at least one pixel that is not in the pattern. Since the classification of each pixel point is isolated, the function ∇f can be parallelized. The pseudocode for the boundary extraction process is shown in Algorithm 1.

Each thread in the GPU kernel is responsible for processing a pixel point in I . The boundary image ∂I is generated by checking the classification of each pixel point in I .

B. Component Analysis

Definition 3 (Connected Component). *A connected component is defined as a set of pixels forming a foreground, where for*

Algorithm 1 Image Boundary Extraction

Input: Image I with shape $[H, W]$.

Output: The boundary image ∂I of I .

```

1:  $block_x, block_y \leftarrow \frac{(H + thread_x - 1)}{thread_x}, \frac{(W + thread_y - 1)}{thread_y}$ ;
2:  $kernel \lll \langle block, thread \rangle \ggg (I, \partial I)$ ;
3: return  $\partial I$ ;
4:
5: function  $kernel(I, \partial I)$ 
6:    $id_x \leftarrow blockIdx.x * blockDim.x + threadIdx.x$ ;
7:    $id_y \leftarrow blockIdx.y * blockDim.y + threadIdx.y$ ;
8:   if  $id_x < H$  and  $id_y < W$  then
9:     if  $I[id_x][id_y] = 1$  and  $I[id_x][id_y] \in \mathcal{B}$  then
10:       $\partial I[id_x][id_y] = 1$ ;
11:     else
12:       $\partial I[id_x][id_y] = 0$ ;

```

Algorithm 2 Parallel Component Analysis

Input: Boundary image ∂I with shape $[H, W]$.

Output: Labeled image $\partial I'$ with shape $[H, W]$.

```

1: Initial: Empty tensor  $\partial I'$ ;
2:  $block_x, block_y \leftarrow \frac{(H + thread_x - 1)}{thread_x}, \frac{(W + thread_y - 1)}{thread_y}$ ;
3:  $kernel \lll \langle block, thread \rangle \ggg (\partial I, \partial I')$ ;
4: return  $\partial I'$ ;
5: // Find the label of the pixel  $(x, y)$  with path compression
6: function  $find(\partial I', (x, y))$ 
7:    $x_o, y_o \leftarrow x, y$ ;
8:   while  $\partial I'[x][y] \neq \text{Index}(x, y)$  do
9:      $x, y \leftarrow \text{Index}^{-1}(\partial I'[x][y])$ ;
10:   $\partial I'[x_o][y_o] \leftarrow \text{Index}(x, y)$ ;
11:  return  $\text{Index}(x, y)$ ;
12: // Merge two labels under atomic lock
13: function  $union(\partial I', (x_i, y_i), (x_j, y_j))$ 
14:   $done \leftarrow false$ ;
15:  while  $done = false$  do
16:     $label_i \leftarrow find(\partial I', (x_i, y_i))$ ;
17:     $label_j \leftarrow find(\partial I', (x_j, y_j))$ ;
18:    if  $label_i < label_j$  then
19:       $x_j, y_j \leftarrow \text{Index}^{-1}(label_j)$ ;
20:       $\partial I'[x_j][y_j] \leftarrow \text{atomicMin}(\partial I'[x_j][y_j], label_i)$ ;
21:    else if  $label_i > label_j$  then
22:       $x_i, y_i \leftarrow \text{Index}^{-1}(label_i)$ ;
23:       $\partial I'[x_i][y_i] \leftarrow \text{atomicMin}(\partial I'[x_i][y_i], label_j)$ ;
24:    else
25:       $done \leftarrow true$ ;
26:
27: function  $kernel(\partial I, \partial I')$ 
28:   $id_x \leftarrow blockIdx.x * blockDim.x + threadIdx.x$ ;
29:   $id_y \leftarrow blockIdx.y * blockDim.y + threadIdx.y$ ;
30:  if  $id_x < H$  and  $id_y < W$  then
31:     $\partial I'[id_x][id_y] \leftarrow \text{Index}(id_x, id_y)$ ; // Initialize the label
32:     $\text{cudaSyncThreads}()$ ;
33:    for  $(x, y) \in \text{neighbors of } (id_x, id_y)$  do
34:      if  $\partial I[x][y] = \partial I[id_x][id_y]$  then
35:         $union(\partial I', (id_x, id_y), (x, y))$ ;

```

any two pixels (x_i, y_i) and (x_j, y_j) in this set, there exists a path composed of pixels that also belong to the set.

Theorem 2. Pixel (x_i, y_i) and pixel (x_j, y_j) belong to the same connected component if and only if there exists a path composed of pixels that lie entirely within the foreground.

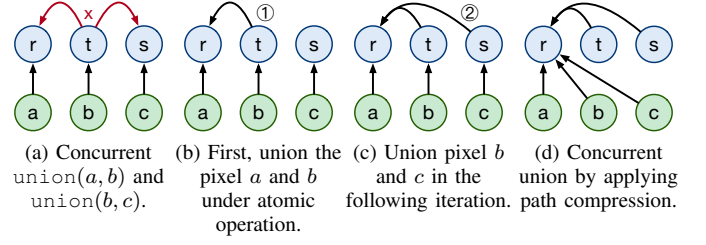


Fig. 3 An illustration of the union function implemented using atomic operations.

Lemma 1. Each boundary in the boundary image ∂I is a connected component.

Definition 3 shows the definition of the connected component. Based on Lemma 1, the goal of the component analysis is to identify all connected components in the boundary image ∂I .

Union-Find Structure on GPU. According to Theorem 2, connected components can be identified by initially grouping connected pixels and then merging sets that belong to the same component. This type of problem can be efficiently solved using the union-find data structure.

The union-find structure maintains a label image $\partial I'$, where each pixel in $\partial I'$ is assigned a unique label. There are two functions in the union-find structure: find and union.

- $find(x_i, y_i)$: Returns the label of the component to which the pixel (x_i, y_i) belongs.
- $union((x_i, y_i), (x_j, y_j))$: Joins the components containing (x_i, y_i) and (x_j, y_j) with the smaller label.

The root pixel in $\partial I'$ is defined as the pixel whose label points to its own index value, represented as a one-dimensional flattened index that locates it within the image. The conversion between the two-dimensional coordinates (x, y) and the one-dimensional index is defined by the functions Index and Index^{-1} .

The implementation of the functions find and union is shown in Algorithm 2. Atomic operations are necessary for parallel union. Fig. 3 illustrates an example of concurrent mergings, where pixels a , b , and c have roots r , t , and s , respectively. The labels of r , s , and t are l_r , l_s , and l_t with $l_r < l_s < l_t$. As shown in Fig. 3(a), if the union functions $union(a, b)$ and $union(b, c)$ are executed simultaneously, the label of b may be overwritten by either l_r or l_s , causing a conflict in the union operation. By using the atomic operation atomicMin , we can ensure a valid execution order as shown in Fig. 3(b) and Fig. 3(c), first merging a and b , and then merging b and c . As a result, the final label of a , b , and c will be l_r .

The path compression optimization in the find function flattens the tree structure of the union-find data structure, significantly improving the efficiency of subsequent queries which is shown in Fig. 3(d).

Kernel Implementation. Algorithm 2 illustrates the algorithmic flow of parallel component analysis. Initially, each component's label is set to its own index value. The algorithm

then iterates through each pixel in the image, examining its neighbors to determine whether they belong to the same component. If a neighboring pixel is found to be part of the same component, the algorithm merges the two labels using the `union` function.

Since the algorithm queries the label image $\partial I'$ multiple times, shared memory is used to store the tiled data of $\partial I'$. This optimization significantly reduces total memory access latency. For initialization, the initial label of each pixel can be set to the smallest index among its neighbors, which helps reduce the number of `union` function calls during the merging process.

After the global union, the labels may not be consecutive numbers, a parallel reduction operation can be used to collect all the unique labels as the label tensor L in the label image $\partial I'$.

C. Starting Point Selection

In Section II-C, we introduced the fundamental concepts of search-based contour tracing algorithms. The searching algorithm starts with a corner pixel on the boundary, which serves as a vertex on the contour, to initiate the tracing process. For each component in the labeled image $\partial I'$, a starting point must be identified to initiate the contour tracing process.

In this process, the key step is identifying an inflection point as the starting point for each component. The definition of an inflection point is shown in Definition 4.

Definition 4. An *inflection point* is defined as a pixel on the boundary o , where the direction of its neighbors changes.

Without loss of generality, we can assume that inflection points have only two possible directions across the pixel. We define the directional edges of an inflection point as unit direction pointers that extend from the point itself to its two neighboring pixels. The inflection point has the following Lemma 2:

Lemma 2. An inflection point on the component is a vertex on the contour.

After obtaining one inflection point as the starting point, we can identify other inflection points sequentially using the searching method to extract the contour.

Algorithm 3 outlines the procedure for selecting starting points for each component in the labeled image $\partial I'$. The algorithm processes each pixel in the labeled image by launching a kernel with a 2D grid of threads. The algorithm will first check whether this pixel is an inflection point. Equation (3) presents the mathematical constraints that define an inflection point.

$$\begin{aligned} & \forall |\delta x + \delta x'| + |\delta y + \delta y'| = 0 \\ \text{s.t. } & \partial I'(x + \delta x, y + \delta y) \neq \partial I'(x + \delta x', y + \delta y'), \end{aligned}$$

where $(\delta x, \delta y)$ is the unit displacement of one direction, and $(\delta x', \delta y')$ is the unit displacement of the opposite direction. The inflection point checking process can be performed using a rational method that examines the neighbors of the pixel in four opposite direction pairs. If the starting point for the corresponding component has not yet been assigned, the pixel's

coordinates are stored in the starting points tensor P , and the corresponding label is marked as visited. The starting points tensor P is a 2D tensor with shape $[N, 2]$, where N represents the number of components in the labeled image $\partial I'$. To guarantee that each component is assigned a unique starting point, the selection process is performed atomically. The starting points tensor plays a critical role in the subsequent contour tracing process, as it provides the initial coordinates required for tracing each contour.

D. Parallel Searching

After obtaining the starting points for each component, we perform contour tracing on each component to transform it into the polygon representation format, as discussed in Section II-B. We introduce a new searching algorithm that can automatically determine the initial searching direction and handle holes simultaneously.

To design the parallel algorithm, it is unnecessary to analyze each pixel in the image, making 2D grid parallelism redundant. Instead, 1D grid parallelism can be employed to process each component in the image. The parallel searching algorithm is designed to handle each component independently, ensuring efficient contour tracing.

Initial Searching Direction. Since the starting point is randomly selected, it is necessary to determine the next point in the counterclockwise movement for each starting point as well as the initiate searching direction, accounting for different turning directions. For the same type of inflection point, the searching directions may differ. As shown in Fig. 4(a) and Fig. 4(b), both inflection points turn in the upward and rightward directions. However, one is a convex point while the other is a concave point, resulting in different searching directions for these two points. To account for the differences caused by the geometric

Algorithm 3 Random Starting Points Selection

Input: Labeled image $\partial I'$ with shape $[H, W]$, label tensor L with shape $[N, 1]$.

Output: Starting points tensor P with shape $[N, 2]$

```

1: Initial:  $visited \leftarrow \{\}$ ;
2:  $block_x, block_y \leftarrow \frac{(H + thread_x - 1)}{thread_x}, \frac{(W + thread_y - 1)}{thread_y}$ ;
3:  $kernel \ll \langle block, thread \rangle \gg (\partial I', L)$ ;
4: return  $\partial I$ ;
5:
6: function  $kernel(\partial I', L)$ 
7:    $id_x \leftarrow blockIdx.x * blockDim.x + threadIdx.x$ ;
8:    $id_y \leftarrow blockIdx.y * blockDim.y + threadIdx.y$ ;
9:   if  $id_x < H$  and  $id_y < W$  then
10:     $label \leftarrow \partial I'[id_x][id_y]$ ;
11:    // Check if the pixel is a inflection point
12:    for  $(d, d') \in$  all opposite directions do
13:       $pre_x, pre_y \leftarrow id_x + d_x, id_y + d_y$ ;
14:       $sub_x, sub_y \leftarrow id_x + d'_x, id_y + d'_y$ ;
15:      if  $\partial I'[pre_x][pre_y] = \partial I'[sub_x][sub_y] = label$  then
16:        return ;
17:   for  $i \leftarrow 0$  to  $N$  do
18:     if  $label = L[i]$  and  $L[i] \notin visited$  then
19:        $P[i] \xleftarrow{\text{atomic}} (id_x, id_y)$ ;
20:        $visited \leftarrow visited \cup L[i]$ ;

```

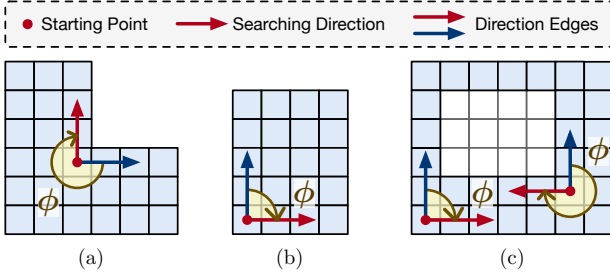


Fig. 4 Illustration of the rotation ϕ of the foreground and the initial searching direction.

characteristics of the original pattern, we define the rotation ϕ of the foreground as Definition 5:

Definition 5. The *rotation* ϕ of the foreground is defined as a circular transformation, where the source moves clockwise from one directional edge of the inflection point, across the foreground, to the other directional edge.

As shown in Fig. 4, the curve array represents the rotation ϕ of each inflection point, which differs even though they have the same directional edges. This indicates that the rotation ϕ of the foreground can be used to determine the search direction. Thus, we can define the initial searching direction as Lemma 3:

Lemma 3. The initial searching direction is defined as the target direction of the foreground rotation ϕ .

Proof. Let v_0 be the starting point, and suppose the two directional edges of v_0 are e_0 and e_1 . Without loss of generality, we assume that the rotation ϕ of each foreground is defined as $e_0 \rightarrow e_1$. During the search process, the algorithm first enters v_0 through $-e_0$ and then exits v_0 through e_1 . As a result, the foreground will remain on the left-hand side during the search, making the process a counterclockwise search. $\square$

Algorithm 4 presents the pseudocode for the parallel searching algorithm used for contour tracing. `init_direction` is a function that determines the initial searching direction for each starting point. First, we enumerate all possible directional pairs at the starting point to determine its two edge directions. Next, we compute the rotation ϕ of the foreground by examining the neighbors of the starting point and identifying the target edge direction.

After obtaining the initial search direction, the parallel searching process can begin. The search is performed in a breadth-first manner. At each step, the algorithm examines the neighbors of the current pixel in a counterclockwise direction, selecting the first unvisited pixel that belongs to the same component. This greedy strategy ensures that every step is consistently taken in the counterclockwise direction. The algorithm records the search direction at each step and compares it with the direction of the next step. If the direction of the next step differs from the current step, this indicates that the current pixel is an inflection point, representing a vertex on the contour. The algorithm then adds the current pixel to the polygon set p and updates the recorded direction. The process continues along

Algorithm 4 Parallel Searching

Input: Image I and labeled image $\partial I'$ with shape $[H, W]$, label tensor L with shape $[N, 1]$, starting points tensor P with shape $[N, 2]$.

Output: The polygon set $\mathcal{C}$ after contour tracing.

```

1: Initial:  $\mathcal{C} \leftarrow \{\}$ ;
2:  $block \leftarrow \frac{N + thread - 1}{thread}$ ;
3:  $kernel \lll block, thread \ggg (I, \partial I', L, P)$ ;
4: return  $\mathcal{C}$ ;
5:
6: function init_direction( $I, \partial I', s, label$ )
7:   for  $d_1, d_2 \in \text{direction pairs}$  do
8:      $n_x, n_y \leftarrow s_x + d_{1x}, s_y + d_{1y}$ ;
9:      $m_x, m_y \leftarrow s_x + d_{2x}, s_y + d_{2y}$ ;
10:    if  $\partial I'[n_x][n_y] = \partial I'[m_x][m_y] = label$  then
11:      for  $d \in \phi$  do
12:         $x, y \leftarrow s_x + d_x, s_y + d_y$ ;
13:        if  $I[x][y] = 1$  then
14:           $direction \leftarrow d$ ;
15:    return  $direction$ ;
16:
17: function kernel( $I, \partial I', L, P$ )
18:    $id \leftarrow blockIdx.x * blockDim.x + threadIdx.x$ ;
19:    $label \leftarrow L[id]$ ;
20:   if  $id < N$  then
21:     Initial:  $p \leftarrow \{P[id]\}$ ;  $visited \leftarrow \{P[id]\}$ ;
22:      $cur \leftarrow P[id]$ ;  $next_x, next_y \leftarrow -1, -1$ ;
23:      $direction \leftarrow \text{init\_direction}(I, \partial I', P[id], label)$ ;
24:     while  $next_x \neq P[id]_x$  or  $next_y \neq P[id]_y$  do
25:       for  $d \in \text{counterclockwise directions}$  do
26:          $new_x, new_y \leftarrow cur_x + d_x, cur_y + d_y$ ;
27:         if  $\partial I'[n_x][n_y] = label$  and  $new \notin visited$  then
28:            $next_x, next_y \leftarrow new_x, new_y$ ;
29:            $visited \leftarrow visited \cup next$ ;
30:            $direction' \leftarrow d$ ;
31:         break;
32:       if  $direction' \neq direction$  then
33:          $p \leftarrow p \cup (next_x, next_y)$ ;
34:          $direction \leftarrow direction'$ ;
35:        $cur_x, cur_y \leftarrow next_x, next_y$ ;
36:    $\mathcal{C} \leftarrow \mathcal{C} \cup p$ ;

```

the boundary until the algorithm returns to the starting point. After completing the parallel searching process, we obtain the polygon set $\mathcal{C}$, which contains all the contours of the image I represented in a counterclockwise format.

Hole Detection. As mentioned in Section II-B, polygons can represent graphics with holes. In our parallel searching algorithm, the hole detection process is seamlessly integrated with the contour tracing process. The algorithm identifies holes by checking the rotation ϕ of the foreground, ensuring adherence to the left-hand rule. As illustrated in Fig. 4(c), the hole contours are traced in a clockwise direction, while the outer contours are traced in a counterclockwise direction. This demonstrates that our searching algorithm is capable of handling complex topological structures.

E. Multi-GPU Solution For Large Layouts

The G-Contour framework can be extended to multi-GPU using a divide-and-conquer approach, enabling it to efficiently process large-scale layouts.

TABLE I Benchmark statistics and runtime comparison of contour tracing using OpenCV [25] and G-Contour. ‘#Pattern’ refers to the number of patterns in the layout, and ‘Avg. Degree’ refers to the average number of vertices in the patterns.

Benchmark	Layer	Layout Size (μm^2)	#Pattern	Avg. Degree	Contour Tracing Runtime (s)		
					OpenCV [25]	G-Contour	Speedup
gcd	Metal	35.70	9722	20.66	2.05	0.22	9.36×
	Via	35.53	8171	10.90	0.86	0.19	4.47×
	Poly	34.97	4385	23.33	0.91	0.55	1.65×
	Activate	35.59	2709	17.20	0.70	0.47	1.48×
ibex	Metal	249.45	89150	56.81	42.29	6.16	6.86×
	Via	255.33	394404	10.42	168.90	9.86	17.12×
	Poly	255.33	81776	24.13	12.08	2.12	5.69×
	Activate	254.72	277139	21.68	454.42	13.95	32.58×
riscv	Metal	176.29	663180	11.69	1566.19	15.94	98.26×
	Via	152.57	170602	5.56	11.32	1.80	6.29×
	Poly	147.11	670730	8.59	1193.35	14.19	84.11×
	Activate	147.25	433960	6.75	136.74	6.09	22.45×
Average					26.64	2.46	10.83×

Lemma 4. Given a layout I , partition it into N sub-layouts $I_1, I_2, \dots, I_N$. Let the contours of these sub-layouts be $\mathcal{C}_1, \mathcal{C}_2, \dots, \mathcal{C}_N$. Then, the contour of the layout I is defined as the union of the contours of all sub-layouts, i.e., $\mathcal{C} = \mathcal{C}_1 \cup \mathcal{C}_2 \cup \dots \cup \mathcal{C}_N$.

Lemma 4 guarantees that the contours of the entire layout can be obtained by merging the contours of the sub-layouts. Based on this, we can employ a producer-consumer strategy to partition the layout into multiple sub-layouts, assigning each sub-layout to a separate GPU for parallel processing. After the individual processing of each sub-layout, the contours are aggregated and merged into a single contour set on the CPU using a polygon merging operation with a complexity of $O(n \log n)$. This final step produces the contours representing the entire layout.

V. EXPERIMENTAL RESULTS

In this section, we evaluate the performance of G-Contour across various benchmarks, comparing it with a state-of-the-art CPU-based contour tracing framework. Experiments are performed on an Ubuntu 20.04 system equipped with an Intel(R) Xeon(R) Gold 6326 CPU @ 2.90GHz, 128GB of main memory, and an NVIDIA A800 GPU with 80GB of memory. G-Contour is implemented in C++ and CUDA and comprises approximately 1,000 lines of code.

For comparison, we employ the open-source contour tracing framework from OpenCV [17], a widely used library for computer vision and image processing, as the baseline. Specifically, we use its `cv::findContours` API, running single-threaded on the CPU, to extract contours from layout images. The algorithm implemented by this API is described in Section II-C.

To demonstrate the applications of G-Contour in processing VLSI layouts, we integrate G-Contour into the state-of-the-art full-chip scale ILT framework FuILT [26]. It is used to extract mask contours from the optimized mask patterns during the ILT optimization process, as discussed in Section I. For authenticity, all benchmarks used in our evaluation are from real industry designs in GDSII format. All benchmarks are generated using the OpenROAD [27], [28] and FreePDK45 [29] process design kit (PDK). The benchmarks include the following:

- **gcd** [27]: A circuit that computes the greatest common divisor of two input numbers.
- **ibex** [30]: An open-source RISC-V core designed for efficient embedded applications.
- **riscv**: A CPU core for the 32-bit RISC-V instruction set architecture capable of booting the Linux kernel.

We further extract different layers of the benchmarks. These layers have varying layout sizes and geometric complexities, as shown in TABLE I, which are sufficient to demonstrate the capability of G-Contour to handle large layouts and complex patterns.

A. Performance Evaluation

We first evaluate the performance of G-Contour on the mask contour extraction task. For each benchmark, the mask is generated by FuILT using the Levelset-based ILT method [9] and subsequently rasterized with a pixel size of $7nm$. TABLE I summarizes the geometric statistics of the benchmarks alongside a runtime comparison between OpenCV and G-Contour for contour tracing. Compared to OpenCV, G-Contour achieves an average speedup of 10.83× across all benchmarks and outperforms OpenCV in every instance, highlighting its effectiveness in accelerating contour tracing. For the `gcd` design, characterized by a small layout size and a limited number of patterns, the CPU implementation of OpenCV exhibits reasonable efficiency; the speedup achieved by G-Contour on the Poly and Activate layers is approximately 1.5×. Conversely, for large-scale layouts such as `riscv`, which contain numerous patterns, the speedup is substantial. As mentioned in Section II-C, OpenCV’s contour tracing algorithm relies on topological structure analysis, which demands considerable processing time for complex patterns. In contrast, G-Contour utilizes parallel pixel-level component analysis and search, enabling the efficient simultaneous handling of complex patterns. As detailed in TABLE I, the speedup of G-Contour on the Metal and Poly layers of `riscv` reaches 98.26× and 84.11×, respectively.

Pattern Complexity v.s. Runtime. As observed in TABLE I, the contour tracing runtime does not scale directly with layout size, and the speedup provided by G-Contour varies significantly across benchmarks. For example, the Via and Activate

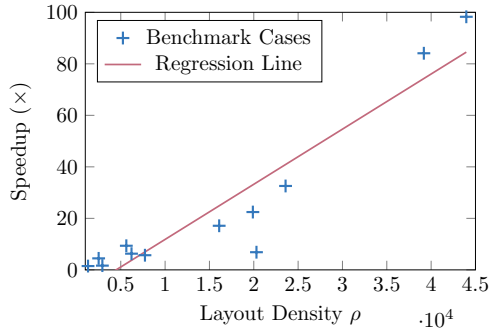


Fig. 5 The relationship between the layout density ρ and the speedup of G-Contour across all benchmarks.

TABLE II Runtime comparison of component analysis between the CPU implementation in OpenCV [25] and Algorithm 2 in G-Contour.

Benchmark	Layer	Component Analysis Runtime (s)		
		OpenCV [25]	G-Contour	Speedup
gcd	Metal	0.0247	0.0053	4.66×
	Via	0.0212	0.0043	4.96×
	Poly	0.0254	0.0045	5.59×
	Activate	0.0221	0.0044	5.02×
ibex	Metal	0.3048	0.0484	6.30×
	Via	0.3423	0.0699	4.89×
	Poly	0.3127	0.0505	6.19×
	Activate	0.3576	0.0493	7.26×
riscv	Metal	0.1955	0.0246	7.94×
	Via	0.1381	0.0177	7.80×
	Poly	0.1294	0.0167	7.76×
	Activate	0.1275	0.0164	7.79×
Average		0.1668	0.0260	6.35×

layers of the *ibex* design contain substantially more patterns than its other layers, resulting in G-Contour speedups of 17.12× and 32.58×, respectively. These figures are considerably higher than the speedups achieved on the other two layers. This variation suggests that G-Contour’s runtime is strongly influenced by the layout’s pattern complexity, which depends on the number of patterns, their average degree, and the layout size. To quantify the relationship between the runtime of G-Contour and the complexity of the patterns, we define the layout density ρ as presented in Equation (4):

$$\rho = \frac{\# \text{Pattern} \times \text{Avg. Degree}}{\text{Layout Size}}. \quad (4)$$

The layout density ρ represents the average number of vertices per unit area in the layout. Fig. 5 illustrates the regression line between the speedup of G-Contour and the layout density ρ . The figure reveals a positive correlation: as ρ increases, the speedup of G-Contour relative to OpenCV also increases. This occurs because higher density corresponds to more patterns within the layout, enabling greater parallelism in the parallel searching step (Algorithm 4). These findings underscore G-Contour’s capability to efficiently manage large-scale, complex layouts compared to OpenCV, highlighting its suitability for real-world VLSI design applications.

TABLE III Runtime Breakdown Analysis of G-Contour. The runtime breakdown is derived based on the average runtime across all benchmarks.

	Boundary Extraction	Component Analysis	Starting Point Selection	Parallel Searching
Ratio (%)	0.0227	1.0149	4.9596	94.0028

Performance of Component Analysis. In Section IV-B, we propose a parallel component analysis algorithm to identify individual pattern boundaries within the layout image. To evaluate its efficiency, we compare its runtime against the GPU implementation of component analysis available in OpenCV [31]. The results, presented in TABLE II, demonstrate that G-Contour achieves an average speedup of 6.35× across all benchmarks, with the speedup exceeding 4.5× for every individual benchmark. In contrast to contour tracing, where runtime often depends on pattern complexity, our component analysis algorithm exhibits relatively stable performance across benchmarks. This stability stems from its pixel-level operation, processing the image pixel-wise, making its runtime largely independent of the geometric complexity of the patterns.

B. Runtime Breakdown Analysis

To gain deeper insights into the runtime of G-Contour, we perform a runtime breakdown analysis to identify the contribution of each G-Contour component to the total execution time. As detailed in TABLE III, the parallel searching algorithm dominates the runtime, accounting for 94.00% of it. This occurs because the parallelism level of the searching algorithm is directly influenced by the number of patterns in the layout, which varies across different layouts. Furthermore, the imbalance in pattern complexity (degree) significantly impact the search performance. Threads assigned to process more complex patterns require longer execution times, leading to workload imbalance that affects the overall efficiency of the parallel searching stage.

VI. CONCLUSION

In this paper, we introduce G-Contour, a GPU-based framework designed for efficient contour tracing of large-scale layouts in VLSI design. G-Contour incorporates union-find-based component analysis and parallel searching algorithms to perform contour tracing in parallel on GPUs. Experimental results on common layout processing tasks related to design for manufacturing demonstrate that G-Contour achieves an average speedup of over 10× and a peak speedup of 98× on specific designs, compared to a state-of-the-art CPU-based implementation. These results establish G-Contour as a highly efficient tool for large-scale layout processing. We anticipate that G-Contour will significantly enhance the efficiency of contour tracing in VLSI design and related fields, potentially fostering further advancements in GPU-based image processing.

ACKNOWLEDGEMENTS

The project is supported in part by Research Grants Council of Hong Kong SAR (No. RFS2425-4S02 and No. CUHK14201624).

REFERENCES

- [1] S. Gupta and S. Kar, "Algorithms to speed up contour tracing in real time image processing systems," *IEEE Access*, vol. 10, pp. 127 365–127 376, 2022.
- [2] J. Seo, S. Chae, J. Shim, D. Kim, C. Cheong, and T.-D. Han, "Fast contour-tracing algorithm based on a pixel-following method for image sensors," *Sensors*, vol. 16, no. 3, p. 353, 2016.
- [3] J. Yang, B. Price, S. Cohen, H. Lee, and M.-H. Yang, "Object contour detection with a fully convolutional encoder-decoder network," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 193–202.
- [4] D. Linsley, J. Kim, A. Ashok, and T. Serre, "Recurrent neural circuits for contour detection," *arXiv preprint arXiv:2010.15314*, 2020.
- [5] H. Yang, W. Zhong, Y. Ma, H. Geng, R. Chen, W. Chen, and B. Yu, "VLSI mask optimization: From shallow to deep learning," in *Proc. ASPDAC*, 2020, pp. 434–439.
- [6] A. Awad, A. Takahashi, S. Tanaka, and C. Kodama, "A fast process variation and pattern fidelity aware mask optimization algorithm," in *Proc. ICCAD*, 2014, pp. 238–245.
- [7] J. Kuang, W.-K. Chow, and E. F. Y. Young, "A robust approach for process variation aware mask optimization," in *Proc. DATE*, 2015, pp. 1591–1594.
- [8] H. Yang, S. Li, Y. Ma, B. Yu, and E. F. Young, "GAN-OPC: Mask optimization with lithography-guided generative adversarial nets," in *Proc. DAC*, 2018, pp. 131:1–131:6.
- [9] Z. Yu, G. Chen, Y. Ma, and B. Yu, "A GPU-enabled level set method for mask optimization," in *Proc. DATE*, 2021.
- [10] G. Chen, W. Chen, Y. Ma, H. Yang, and B. Yu, "DAMO: Deep agile mask optimization for full chip scale," in *Proc. ICCAD*, 2020.
- [11] B. Jiang, L. Liu, Y. Ma, H. Zhang, E. F. Y. Young, and B. Yu, "Neural-ILT: Migrating ILT to neural networks for mask printability and complexity co-optimization," in *Proc. ICCAD*, 2020.
- [12] Y. Ding, C. Chu, and X. Zhou, "An efficient shift invariant rasterization algorithm for all-angle mask patterns in ilt," in *Proceedings of the 52nd Annual Design Automation Conference*, 2015, pp. 1–6.
- [13] J. Xu, Z. He, S. Yin, Y. Pu, W. Yu, and B. Yu, "Easymrc: Efficient mask rule checking via representative edge sampling," *ACM Transactions on Design Automation of Electronic Systems*, 2025.
- [14] M. Graphics, "Calibre verification user's manual," 2008.
- [15] "KLayout," <https://www.klayout.de/>, 2024.
- [16] G. Chen, H. Geng, B. Yu, and D. Z. Pan, "Open-source differentiable lithography imaging framework," in *DTCO and Computational Patterning III*, vol. 12954. SPIE, 2024, pp. 118–127.
- [17] G. Bradski, "The opencv library," *Dr. Dobbs' Journal: Software Tools for the Professional Programmer*, vol. 25, no. 11, pp. 120–123, 2000.
- [18] T. Miyatake, H. Matsushima, and M. Ejiri, "Contour representation of binary images using run-type direction codes," *Machine Vision and Applications*, vol. 9, pp. 193–200, 1997.
- [19] S. Suzuki *et al.*, "Topological structural analysis of digitized binary images by border following," *Computer vision, graphics, and image processing*, vol. 30, no. 1, pp. 32–46, 1985.
- [20] E. Gose, R. Johnsonbaugh, and S. Jost, *Pattern recognition and image analysis*. Prentice-Hall, Inc., 1996.
- [21] I. Pitas, *Digital image processing algorithms and applications*. John Wiley & Sons, 2000.
- [22] M. Das, M. J. Paulik, and N. K. Loh, "A bivariate autoregressive technique for analysis and classification of planar shapes," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 12, no. 1, pp. 97–103, 2002.
- [23] T. Pavlidis, *Algorithms for graphics and image processing*. Springer Science & Business Media, 2012.
- [24] P. R. Reddy, V. Amarnadh, and M. Bhaskar, "Evaluation of stopping criterion in contour tracing algorithms," *International Journal of Computer Science and Information Technologies*, vol. 3, no. 3, pp. 3888–3894, 2012.
- [25] G. Bradski, "The OpenCV Library," *Dr. Dobbs' Journal of Software Tools*, 2000.
- [26] S. Yin, W. Zhao, L. Xie, H. Chen, Y. Ma, T.-Y. Ho, and B. Yu, "Fuilt: Full chip ilt system with boundary healing," in *Proceedings of the 2024 International Symposium on Physical Design*, 2024, pp. 13–20.
- [27] T. Ajayi and D. Blaauw, "Openroad: Toward a self-driving, open-source digital layout implementation tool chain," in *Proceedings of Government Microcircuit Applications and Critical Technology Conference*, 2019.
- [28] T. Ajayi, V. A. Chhabria, M. Fogaça, S. Hashemi, A. Hosny, A. B. Kahng, M. Kim, J. Lee, U. Mallappa, M. Neseem *et al.*, "Toward an open-source digital flow: First learnings from the openroad project," in *Proceedings of the 56th Annual Design Automation Conference 2019*, 2019, pp. 1–4.
- [29] "FreePDK45," <https://eda.ncsu.edu/freepdk/freepdk45/>.
- [30] A. Amid, D. Biancolin, A. Gonzalez, D. Grubb, S. Karandikar, H. Liew, A. Magyar, H. Mao, A. Ou, N. Pemberton *et al.*, "Chipyard: Integrated design, simulation, and implementation framework for custom socs," *Ieee Micro*, vol. 40, no. 4, pp. 10–21, 2020.
- [31] F. Bolelli, S. Allegretti, L. Baraldi, and C. Grana, "Spaghetti labeling: Directed acyclic graphs for block-based connected components labeling," *IEEE Transactions on Image Processing*, vol. 29, pp. 1999–2012, 2019.